

Порядок разборки/перевода/сборки для Breed для Xiaomi Mi Router 3G с NAND флеш-памятью

Исходно я наткнулся на пост на форуме OpenWrt на сообщение **dabyd64** [Success on hacking, extracting, modifying \(Translating\) and repacking BREED Bootloader!](#), в нем описан порядок разборки/перевода/сборки Breed для SPI флеш-памяти и всего один патч.

Для более «научеёмких» загрузчиков процесс несколько сложнее, забегаая вперед – подсчёт трёх контрольных сумм двумя разными алгоритмами и пр.

Так же отмечу, что CRC указная как POSIX в статье **johovich** [на Хабре](#) если использовать, например, подсчёт контрольной суммы с помощью `rusrc` с ключом `--model=posix` (<https://pycrc.org/>) или CRC Calculator от soltau (<https://sourceforge.net/projects/crc-calculator/>) подсчитывается то, наверняка, правильно, но этот, видимо не тот «CRC POSIX», что использован в Breed!

Отдельная благодарность **Nicky F.** с форума [4PDA](#) за отзывчивость, за скрипт на Python'е для разборки/сборки Breed!

Я, правда, мало что понял в скрипте, но это всё, потому что я ни разу не программист, а не потому, что скрипт неправильный!

Ссылка на **Breed 1.2 r1416 [2022-07-24] (git-46ae2a1)** для Xiaomi Mi Router 3G

breed-mt7621-xiaomi-r3g.bin (MD5: e65d388129a6d1ac39abf99329f1978b)

<https://breed.hackpascal.net/r1416%20%5b2022-07-24%5d/breed-mt7621-xiaomi-r3g.bin>

<https://breed.hackpascal.net/breed-mt7621-xiaomi-r3g.bin>

1. Инструментарий

A) binwalk Firmware Analysis Tool

<https://github.com/ReFirmLabs/binwalk>

B) u-boot-tools. Точнее dumpimage из пакета u-boot-tools

C) XZ Utils. Точнее lzmainfo из пакета XZ Utils

<https://github.com/tukaani-project/xz>

D) lzma из пакета LZMA SDK. Опытным путем выяснено, что lzma-data в Breed упакованы lzma версией 4.62

<https://sourceforge.net/projects/sevenzips/files/LZMA%20SDK/4.62/>

E) Шестнадцатеричный редактор. Я использовал 010 Editor

<https://www.sweetscape.com/010editor/>

F) HTTP-сервер. Я использовал HTTP File Server (HFS)

<https://github.com/rejetto/hfs>

G) *NIX утилиты: dd, cat, crc32, cksum, printf. Опционально date.

Все утилиты есть в порте BusyBox для Windows

<https://frippersy.org/busybox/>

H) PuTTY

Автор Breed пишет о несовместимости Breed и telnet-клиента в Linux, предлагает использовать PuTTY: «Please use the telnet client that comes with Windows or PuTTY. The telnet client under Linux is not compatible».

<https://putty.sourceforge.io/>

I) Windows + виртуальная машина с Linux для запуска binwalk и dumpimage или просто Linux.

2. Примеры использования команд.

**В Windows всё тоже самое, что и в Linux,
только перед командой надо ввести busybox**

Пример ввода команды crc32 в Windows

```
busybox crc32 04NoHeader-MiR3G-Cn
```

Перевод из десятичной системы в шестнадцатеричную

```
printf %X 1658592583  
62DC1D47
```

Перевод из шестнадцатеричной системы в десятичную

```
printf %d 0x62DC1D47  
1658592583
```

Подсчёт CRC32. Вывод – шестнадцатеричное число

```
crc32 04NoHeader-MiR3G-Cn  
df0a279a 04NoHeader-MiR3G-Cn
```

Подсчёт CRC32 POSIX/cksum. Вывод – десятичное число

```
cksum 01uImage-Header-MiR3G-Cn-Mod1  
26846098 64 01uImage-Header-MiR3G-Cn-Mod1
```

Перевод из десятичной системы в шестнадцатеричную

```
printf %X 26846098  
199A392
```

Перевод даты/времени в десятичное число «Эпохи Unix»

```
date -d "2022-07-23 16:09:43" +%s  
1658592583
```

Перевод десятичного числа «Эпохи Unix» в дату/время

```
date -R -u @1658592583  
Sat, 23 Jul 2022 16:09:43 +0000
```

**Перевод текущей (на момент исполнения команды) даты/времени
в десятичное число «Эпохи Unix»**

```
date -R -u +%s
```

**Вырезать первые 64 байта из breed-mt7621-xiaomi-r3g.bin и сохранить в файл
01uImage-Header-MiR3G-Cn**

```
dd if=breed-mt7621-xiaomi-r3g.bin of=01uImage-Header-MiR3G-Cn bs=1 count=64 skip=0
```

**Отступить первые 64 байта в breed-mt7621-xiaomi-r3g.bin, вырезать кусок размером
28156 байт и сохранить в файл 02Boot-MiR3G-Cn**

```
dd if=breed-mt7621-xiaomi-r3g.bin of=02Boot-MiR3G-Cn bs=1 count=28156 skip=64
```

**Отступить от начала файла 28220 байта и сохранить оставшийся кусок в файл
03Data-MiR3G-Cn.lzma**

```
dd if=breed-mt7621-xiaomi-r3g.bin of=03Data-MiR3G-Cn.lzma bs=1 skip=28220
```

Склеить последовательно файлы 01uImage-Header-MiR3G-En, 02Boot-MiR3G-En, 03Data-MiR3G-En.lzma и записать в файл breed-mt7621-xiaomi-r3g-En-Test.bin

```
cat 01uImage-Header-MiR3G-En 02Boot-MiR3G-En 03Data-MiR3G-En.lzma >
```

```
breed-mt7621-xiaomi-r3g-En-Test.bin
```

В Windows можно использовать команду copy с ключом /b

```
copy /b 01uImage-Header-MiR3G-En + 02Boot-MiR3G-En + 03Data-MiR3G-En.lzma
```

```
breed-mt7621-xiaomi-r3g-En-Test.bin
```

3. Запуск binwalk, анализ файла Breed

```
binwalk breed-mt7621-xiaomi-r3g.bin
```

DECIMAL	HEXADECIMAL	DESCRIPTION
---------	-------------	-------------

0	0x0	uImage header, header size: 64 bytes, Header CRC: 0x2CD9C16A, created: 2022-07-23 16:09:43, image size: 137186 bytes, Data Address: 0xA0201000, Entry Point: 0xA0201000, Data CRC: 0xDF0A279A, OS: Linux, CPU: MIPS, image type: Standalone Program, compression type: none, image name: "Breed MT7621"
24213	0x5E95	JB00T STAG header, image id: 7, timestamp 0x5004418, image size: 1074814976 bytes, image JB00T checksum: 0x218, header JB00T checksum: 0x2100
27552	0x6BA0	Copyright string: "Copyright (C) 2022 HackPascal <hackpascal@gmail.com>"
28220	0x6E3C	LZMA compressed data, properties: 0x6D, dictionary size: 33554432 bytes, uncompressed size: 373234 bytes

4. Разрезание файла Breed

```
dd if=breed-mt7621-xiaomi-r3g.bin of=01uImage-Header-MiR3G-Cn bs=1 count=64 skip=0
```

```
dd if=breed-mt7621-xiaomi-r3g.bin of=02Boot-MiR3G-Cn bs=1 count=28156 skip=64
```

```
dd if=breed-mt7621-xiaomi-r3g.bin of=03Data-MiR3G-Cn.lzma bs=1 skip=28220
```

5. Проверка настроек сжатия. Записываем размер словаря, значения lc, lp и pb

```
lzmainfo.exe 03Data-MiR3G-Cn.lzma
```

```
Uncompressed size: 0 MB (373234 bytes)
```

```
Dictionary size: 32 MB (2^25 bytes)
```

```
Literal context bits (lc): 1
```

```
Literal pos bits (lp): 2
```

```
Number of pos bits (pb): 2
```

6. Распаковка данных на китайском языке

```
lzma d 03Data-MiR3G-Cn.lzma 03Data-MiR3G-Cn
```

7. Редактирование данных. Нюансы перевода Breed

Копируем/переименовываем файл данных из 03Data-MiR3G-Cn 03Data-MiR3G-En
Открываем 03Data-MiR3G-En в шестнадцатеричном редакторе, выставляем кодировку отображения UTF-8.

Крайне желательно включить подсветку управляющих символов.

В 010 Editor:

View → Character Set → UTF-8

View → Highlighting → Control Characters, Zeros

Первые слова на китайском языке находятся рядом со строкой «BREED:ABORT», чуть ниже.

В Unicode каждый символ китайских иероглифов в кодировке UTF-8 занимает 3 байта (4 байта для редких символов, по-моему, я не встречал в Breed 4-х битные иероглифы).

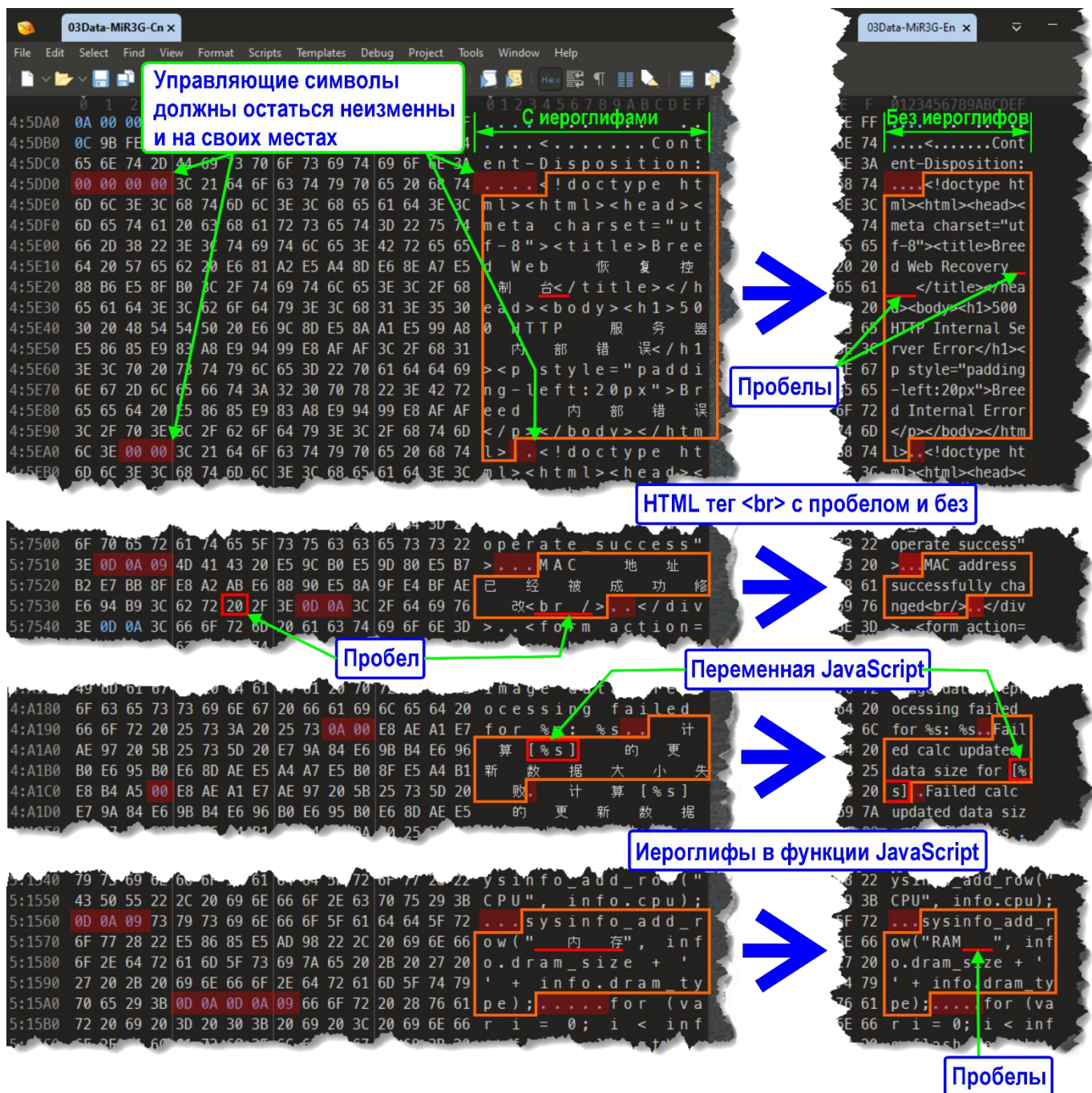
Так же в «китайском Unicode» есть знаки препинания: пробел, точка, запятая, восклицательный знак, двоеточие и т.д.

И, каждый знак китайской пунктуации тоже занимает 3 (три) бита!

Производим замену иероглифов на более понятные символы. Всё делаем неторопливо, аккуратно и внимательно!

- 7.1. Размер файла (в байтах) не должен измениться! Следим за этим.
Делаем периодически резервные копии.
- 7.2. Заменять нужно, ни в коем случае не затрагивая управляющие символы Null [00], HT [09], LF [0A], CR [0D] и т. д.
Все управляющие символы должны остаться в своих значениях и на своих местах!
- 7.3. Не редактируйте строки не содержащие иероглифы, без дизассемблирования доподлинно не известно, что они значат в коде Breed, их повреждение может привести к неработоспособности Breed, то, что называется к окирпчиванию роутера!
- 7.4. Заменяйте исключительно иероглифы на символы ASCII 7 бит (блок Unicode «Основная латиница») – не ошибётесь!
- 7.5. Если необходимо место, то можно получить один дополнительный символ изменив HTML тег содержащий пробел. Например:
 →

- 7.6. Если нет место справа, но есть пробелы слева в пределах HTML тегов, граничащих с управляющими символами, то смещаем HTML теги, строки перевода влево. Все HTML теги надо сохранить!
- 7.7. Если же наоборот – справа от переведенного текста остались иероглифы/один-два бита от иероглифа, то надо их заменить на пробелы.
- 7.8. Будьте внимательны редактируя функции JavaScript содержащие иероглифы и строки содержащие кроме иероглифов переменные JavaScript.
- 7.9. Если перевод не помещается – сокращайте, подбирайте синонимы. Всё одно – будет куда понятнее чем иероглифы!
- 7.10. В 010 Editor ширина части окна, где производим редактирование содержащая китайские иероглифы примерно в полтора раза шире, чем при их отсутствии.
Уменьшение ширина можно считать индикатором конечности процесса перевода.



8. Запаковка данных на английском языке. Используем lzma 4.62

```
lzma e 03Data-MiR3G-En 03Data-MiR3G-En.lzma -d25 -lc1 -lp2 -pb2
```

9. Промежуточная сборка Breed для тестирования без записи во флеш-память роутера

```
cat 01uImage-Header-MiR3G-Cn 02Boot-MiR3G-Cn 03Data-MiR3G-En.lzma >
```

```
breed-mt7621-xiaomi-r3g-En-Test.bin
```

или

```
copy /b 01uImage-Header-MiR3G-Cn + 02Boot-MiR3G-Cn + 03Data-MiR3G-En.lzma
```

```
breed-mt7621-xiaomi-r3g-En-Test.bin
```

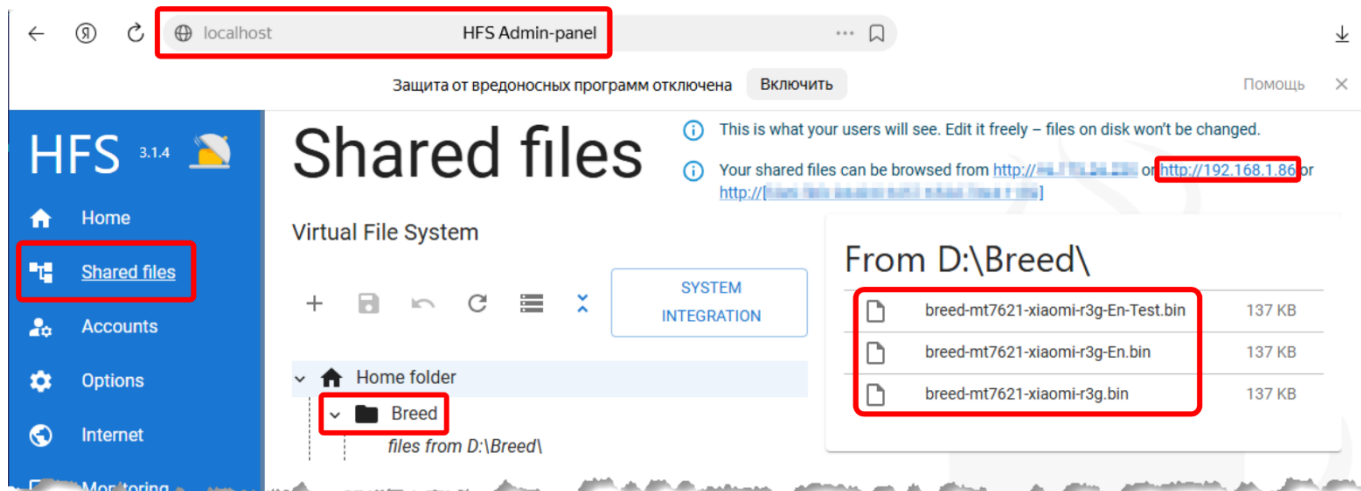
10. Тестирование, проверка результата

10.1. Скачиваем HTTP File Server (HFS).

10.2. Отключаем Wi-Fi

10.3. Запускаем HFS. Admin-panel → Shared files создаем папку, в которой будут доступны файлы по HTTP.

В моем примере папка Breed, IP-адрес 192.168.1.86, имя файла Breed – breed-mt7621-xiaomi-r3g-En-Test.bin



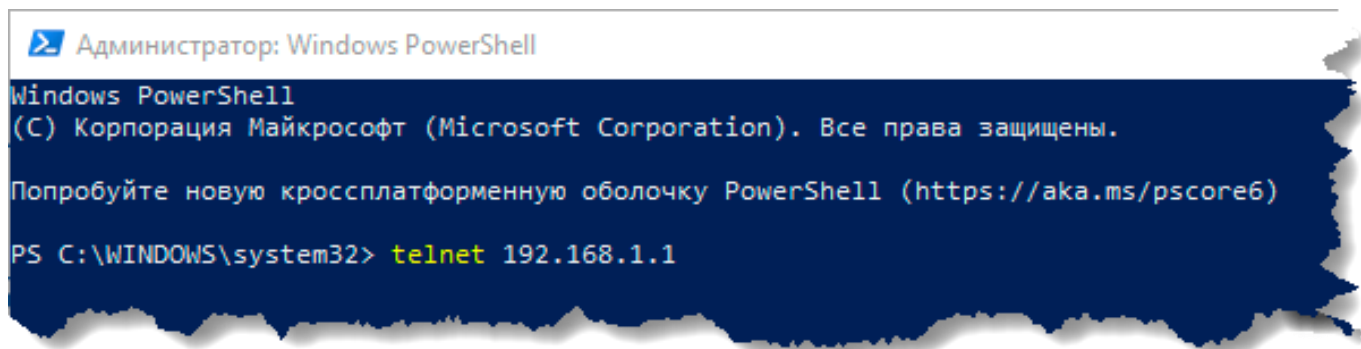
10.4. Выключаем роутер и включаем, удерживая кнопку Reset, ждем 7-10 секунд после того, как светодиоды начнут моргать, отпускаем Reset.

10.5. Соединяем патч-кордом компьютер с роутером.

10.6. Открываем браузер в режиме «Инкогнито» и переходим по адресу 192.168.1.1 в Веб интерфейс Breed.

10.7. Переключаемся в PowerShell. Подключаемся по telnet к роутеру. Вводим в PowerShell команду:

```
telnet 192.168.1.1
```



10.8. Вывод терминала PowerShell:

```
Boot and Recovery Environment for Embedded Devices
Copyright (C) 2022 HackPascal <hackpascal@gmail.com>
Build date 2022-07-24 [git-46ae2a1]
Version 1.2 (r1416)
```

Starting breed built-in shell

breed>

Telnet 192.168.1.1

```
Boot and Recovery Environment for Embedded Devices
Copyright (C) 2022 HackPascal <hackpascal@gmail.com>
Build date 2022-07-24 [git-46ae2a1]
Version 1.2 (r1416)
```

Starting breed built-in shell

breed>

10.9. Вводим в Breed команду:

```
wget http://192.168.1.86/Breed/breed-mt7621-xiaomi-r3g-En-Test.bin
```

Вывод терминала PowerShell:

Connecting to 192.168.1.86:80... connected.

HTTP request sent, awaiting response... 200 OK

Length: 136741/0x21625 (133KB) [application/octet-stream]

Saving to address 0x80001000

[=====] 100%

Transmission completed in 0.2s.

Telnet 192.168.1.1

```
Boot and Recovery Environment for Embedded Devices
Copyright (C) 2022 HackPascal <hackpascal@gmail.com>
Build date 2022-07-24 [git-46ae2a1]
Version 1.2 (r1416)
```

Starting breed built-in shell

breed> wget http://192.168.1.86/Breed/breed-mt7621-xiaomi-r3g-En-Test.bin

Connecting to 192.168.1.86:80... connected.

HTTP request sent, awaiting response... 200 OK

Length: 136741/0x21625 (133KB) [application/octet-stream]

Saving to address 0x80001000

[=====] 100%

Transmission completed in 0.2s.

breed>

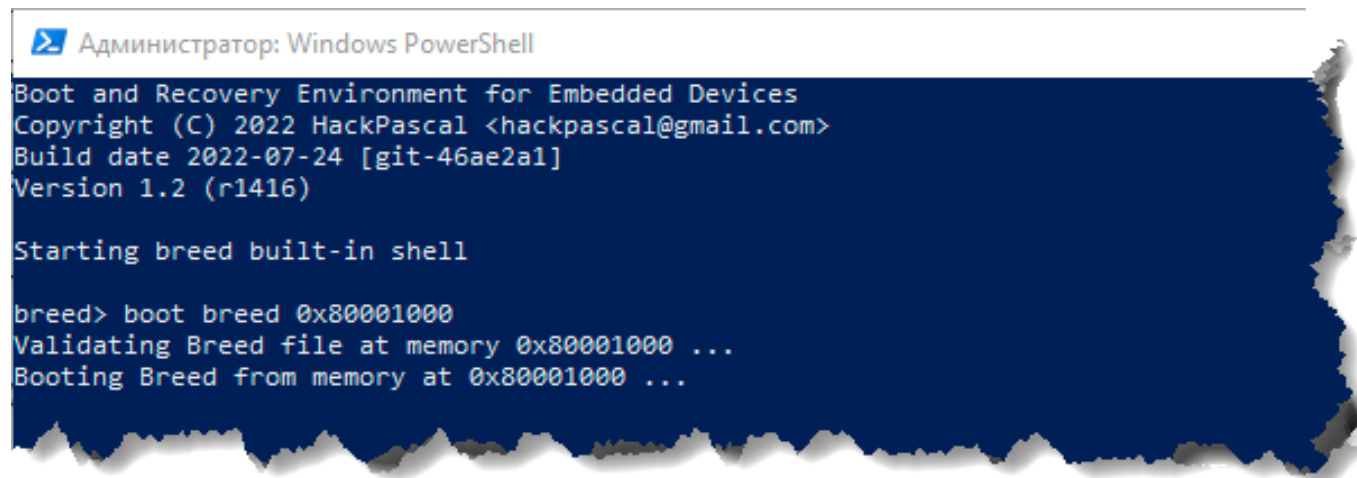
10.10. Далее вводим в Breed команду:

```
boot breed 0x80001000
```

Вывод терминала PowerShell:

```
Validating Breed file at memory 0x80001000 ...
```

```
Booting Breed from memory at 0x80001000 ...
```



10.11. Переключаемся в браузер, обновляем страницу (кнопка F5).

Должен появиться переведенный Breed.

10.12. Смотрим на результаты труда, проверяем все меню, таблицы, чекбоксы и пр. на предмет работоспособности, наличия/отсутствия ошибок, корректности перевода и т. д.

Не пытайтесь прошить в память роутера breed-mt7621-xiaomi-r3g-En-Test.bin.

Бесполезно. Получите сообщение об ошибке на китайском языке в переводе примерно следующее: «Unable define flash layout of [Bootloader] specify manually» или «Data preprocess. [Bootloader] failed: Invalid data».

10.13. Если нашли опечатки/ошибки и т. п. – снова редактируем/запаковываем 03Data-MiR3G-En, снова собираем breed-mt7621-xiaomi-r3g-En-Test.bin, снова wget/boot breed 0x80001000 ...

Если всё в порядке переходим к самому интересному!

11. Про заголовки в загрузчике Breed

ulmage Header. Breed 1.2 r1416 [2022-07-24] (git-46ae2a1)

breed-mt7621-xiaomi-r3g.bin

(MD5: e65d388129a6d1ac39abf99329f1978b)

The image shows a hex dump of the ulmage header for 'breed-mt7621-xiaomi-r3g.bin'. Annotations include:

- #3 CRC32 ulmage Header**: Points to the CRC32 value at offset 00000008 (hex 62 DC 1D 47).
- Размер (Hex) Breed без ulmage Header**: Points to the value at offset 00000004 (hex 00 02 17 E2).
- «Волшебное число» ulmage Header**: Points to the magic number at offset 00000000 (hex 27 05 19 56).
- Дата сборки – Unix Timestamp***: Points to the timestamp at offset 0000000C (hex 62 DC 1D 47).
- 05 OS: Linux**, **05 CPU: MIPS**, **01 Image type: Standalone Program**, **00 Compression type: None**: Points to the metadata fields at offset 00000010.
- Data Load Address**: Points to the address at offset 00000014 (hex 42 72 65 65).
- #1 CRC32 Breed без ulmage Header**: Points to the CRC32 value at offset 00000018 (hex 00 00 00 00).
- #2 CRC32 POSIX/cksum (Hex) ulmage Header**: Points to the cksum value at offset 0000001C (hex 01 99 A3 92).
- Image Name**: Points to the image name 'Breed MT7621...' at offset 00000020.
- Entry Point Address**: Points to the entry point address at offset 00000024 (hex 00 00 00 00).

После Image Name идут еще 4 блока заголовка по 4 байта каждый. Я не знаю, что они значат.

* Дата сборки – Unix Timestamp:
62 DC 1D 47 (Hex) → 1658592583 (Dec)
Команда *NIX "date":
date -R -u @1658592583
Sat, 23 Jul 2022 16:09:43 +0000

Breed 1.2 r1416 [2022-07-24] (git-46ae2a1) Header

breed-mt7621-xiaomi-r3g.bin

(MD5: e65d388129a6d1ac39abf99329f1978b)

The image shows a hex dump of the Breed header for 'breed-mt7621-xiaomi-r3g.bin'. Annotations include:

- Размер (Hex) сжатых данных (архив LZMA)**: Points to the compressed data size at offset 006E10 (hex 00 00 00 00).
- Заголовок архива LZMA**: Points to the LZMA header at offset 006E20 (hex FF FF FF FF).
- «Волшебное число» заголовка Breed**: Points to the magic number at offset 006E30 (hex 37 54 01 78).

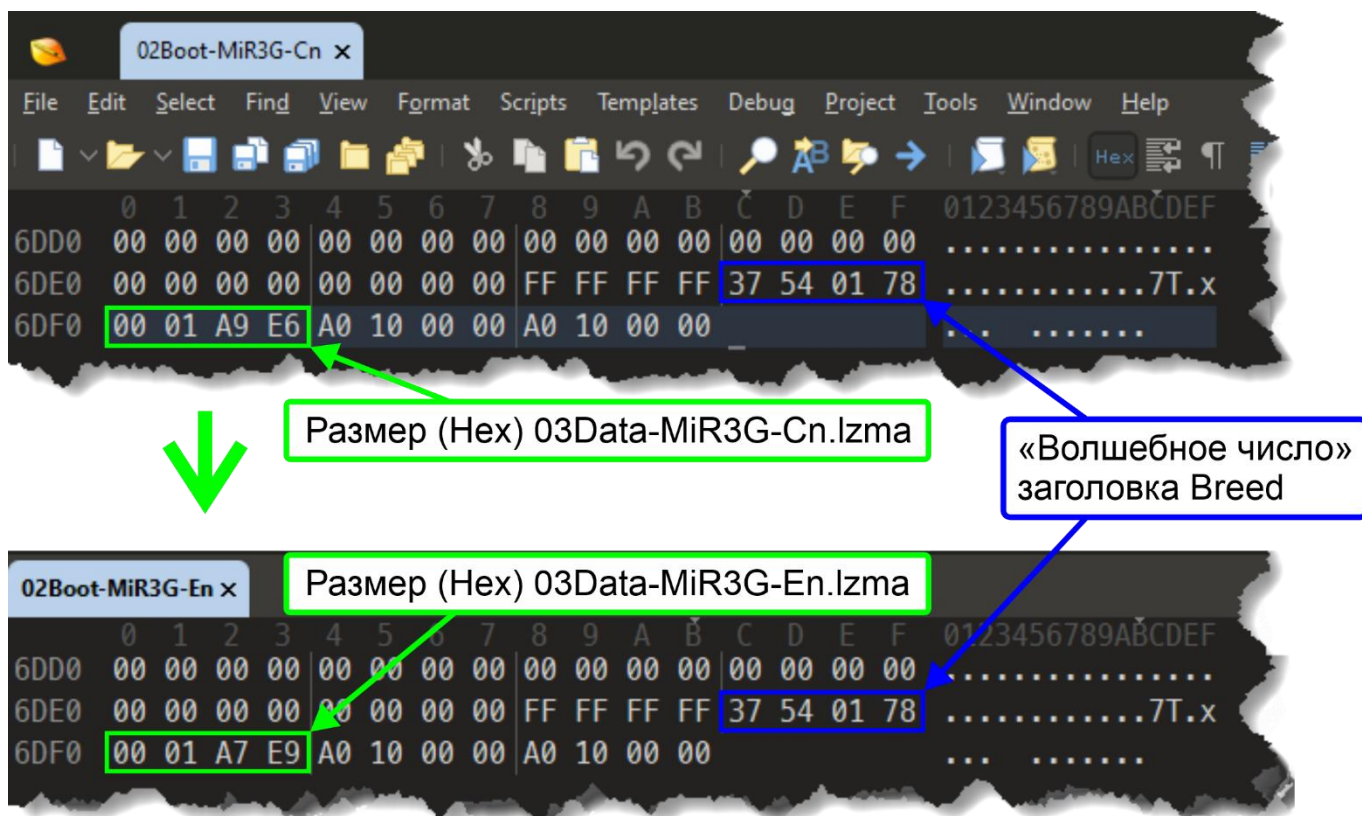
Размер несжатых данных в шестнадцатиричном виде справа налево. Несжатые данные в Breed в байтах:
373234 (Dec) → 00 05 B1 F2 (Hex) → F2 B1 05 00 (Hex RTL)

12. Формирование ulmage Header

- 12.1. Смотрим размер финального 03Data-MiR3G-En.lzma, записываем/запоминаем. В моем примере размер 03Data-MiR3G-En.lzma 108521 байт.
- 12.2. Переводим размер 03Data-MiR3G-En.lzma из десятичной в шестнадцатеричную систему:
`printf %X 108521`
1A7E9
- 12.3. Открываем в шестнадцатеричном редакторе файл 02Boot-MiR3G-Cn. Находим заголовок Breed по шестнадцатеричному значению («волшебное число» заголовка Breed) **37 54 01 78** следующие 4 байта это размер 03Data-MiR3G-Cn.lzma в шестнадцатеричном виде: **00 01 A9 E6**

- 12.4. Заменяем **00 01 A9 E6** на **00 01 A7 E9**.
- ```
0000006DE0: 00 00 00 00 00 00 00 00 FF FF FF FF 37 54 01 78
0000006DF0: 00 01 A9 E6 A0 10 00 00 A0 10 00 00

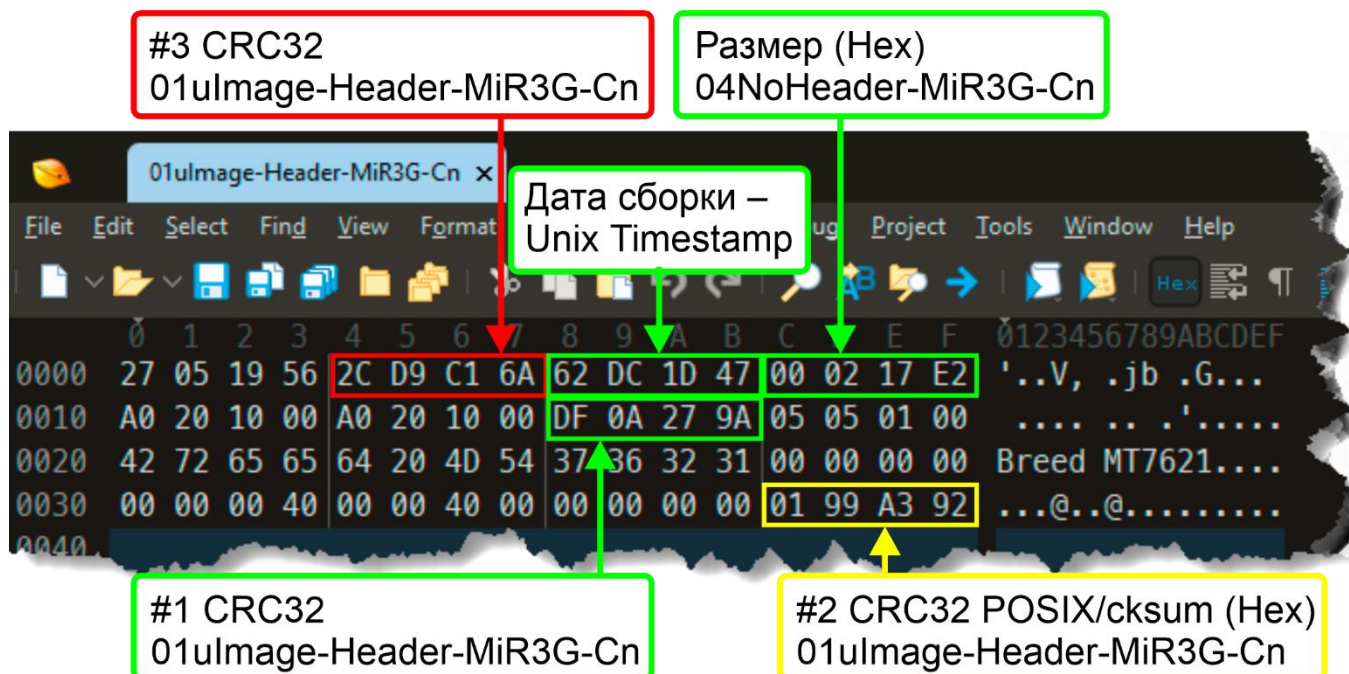
 ↓ ↓ ↓ ↓
0000006DE0: 00 00 00 00 00 00 00 00 FF FF FF FF 37 54 01 78
0000006DF0: 00 01 A7 E9 A0 10 00 00 A0 10 00 00
```



- 12.5. Сохраняем файл с именем 02Boot-MiR3G-En.
- 12.6. Собираем Breed без заголовка ulmage:  
`cat 02Boot-MiR3G-En 03Data-MiR3G-En.lzma > 04NoHeader-MiR3G-En`  
или  
`copy /b 02Boot-MiR3G-En + 03Data-MiR3G-En.lzma 04NoHeader-MiR3G-En`
- 12.7. Смотрим размер 04NoHeader-MiR3G-En, записываем/запоминаем. В моем примере размер 04NoHeader-MiR3G-En 136677 байт.
- 12.8. Переводим размер 04NoHeader-MiR3G-En из десятичной в шестнадцатеричную систему:  
`printf %X 136677`  
215E5

- 12.9. Считаем контрольную сумму 04NoHeader-MiR3G-En по алгоритму CRC32:  
 crc32 04NoHeader-MiR3G-En  
 1b83f99b 04NoHeader-MiR3G-En
- 12.10. Открываем в шестнадцатеричном редакторе файл 01ulmage-Header-MiR3G-Cn
- ```

0000000000: 27 05 19 56 2C D9 C1 6A 62 DC 1D 47 00 02 17 E2
0000000010: A0 20 10 00 A0 20 10 00 DF 0A 27 9A 05 05 01 00
0000000020: 42 72 65 65 64 20 4D 54 37 36 32 31 00 00 00 00
0000000030: 00 00 00 40 00 00 40 00 00 00 00 00 00 00 01 99 A3 92
  
```



Байты с 05 по 08 заменяем нулями:

2C D9 C1 6A → 00 00 00 00

Байты с 13 по 16 заменяем на размер (Hex) 04NoHeader-MiR3G-En:

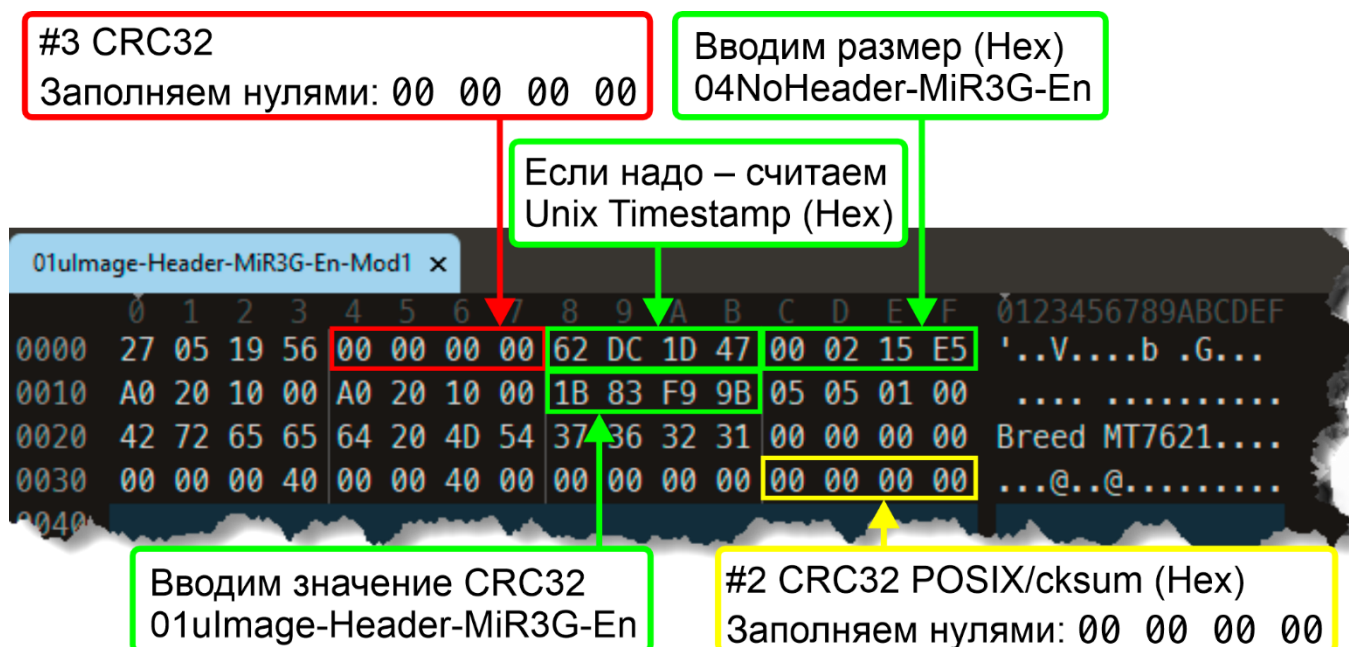
00 02 17 E2 → 00 02 15 E5

Байты с 25 по 28 заменяем на CRC32 04NoHeader-MiR3G-En:

DF 0A 27 9A → 1B 83 F9 9B

Байты с 61 по 64 заменяем нулями:

01 99 A3 92 → 00 00 00 00



12.10.1. Опционно. Я сохранил исходный Timestamp Breed на китайском языке. Байты с 09 по 12 содержат дату сборки – Unix Timestamp, если есть необходимость, то на данном этапе можно в заголовок внести Timestamp. Алгоритм подсчета Unix Timestamp см. раздел **2. Примеры использования команд**. Команда `date`.

Далее переводим десятичное число «Эпохи Unix» в шестнадцатеричное и заносим в заголовок в байты с 09 по 12.

12.11. Сохраняем файл с именем `01ulmage-Header-MiR3G-En-Mod1`

12.12. Считаем контрольную сумму `01ulmage-Header-MiR3G-En-Mod1` по алгоритму CRC32 POSIX/cksum:

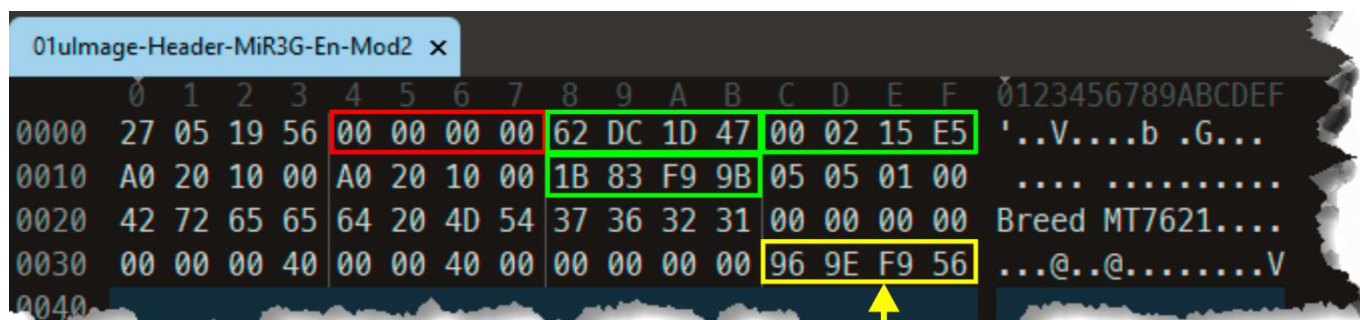
```
cksum 01uImage-Header-MiR3G-En-Mod1
2527000918 64 01uImage-Header-MiR3G-En-Mod1
```

12.13. Переводим 2527000918 из десятичной в шестнадцатеричную систему:

```
printf %X 2527000918
969EF956
```

12.14. Открываем в шестнадцатеричном редакторе файл `01ulmage-Header-MiR3G-En-Mod1`. Байты с 61 по 64 заменяем на CRC32 POSIX/cksum (Hex) `01ulmage-Header-MiR3G-En-Mod1`:

00 00 00 00 → 96 9E F9 56



Вводим значение
CRC32 POSIX/cksum (Hex)
`01ulmage-Header-MiR3G-En-Mod1`

12.15. Сохраняем файл с именем `01ulmage-Header-MiR3G-En-Mod2`

- 12.16. Считаем контрольную сумму 01ulImage-Header-MiR3G-En-Mod2 по алгоритму CRC32:
 crc32 01ulImage-Header-MiR3G-En-Mod2
 7ddb7cad
- 12.17. Байты с 05 по 08 заменяем
 на CRC32 01ulImage-Header-MiR3G-En-Mod2:
00 00 00 00 → 7D DB 7C AD

Вводим значение CRC32
 01ulImage-Header-MiR3G-En-Mod2

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Hex	ASCII
0000	27	05	19	56	7D	DB	7C	AD	62	DC	1D	47	00	02	15	E5	'..V}	..b .G...
0010	A0	20	10	00	A0	20	10	00	1B	83	F9	9B	05	05	01	00		
0020	42	72	65	65	64	20	4D	54	37	36	32	31	00	00	00	00	Breed	MT7621....
0030	00	00	00	40	00	00	40	00	00	00	00	00	96	9E	F9	56	...@..@.....	V
0040																		

- 12.18. Сохраняем файл с именем 01ulImage-Header-MiR3G-En

13. Финальная сборка образа Breed

```
cat 01ulImage-Header-MiR3G-En 04NoHeader-MiR3G-En > breed-mt7621-xiaomi-r3g-En.bin
```

или

```
copy /b 01ulImage-Header-MiR3G-En + 04NoHeader-MiR3G-En breed-mt7621-xiaomi-r3g-En.bin
```


14. Проверка breed-mt7621-xiaomi-r3g-En.bin

- 14.1. Проверяем собранный файл Breed с помощью binwalk. Обращаем внимание на значения вывода binwalk Header CRC (байты заголовка с 05 по 08) и Data CRC (байты заголовка с 25 по 28)

```
binwalk breed-mt7621-xiaomi-r3g-En.bin
```

DECIMAL	HEXADECIMAL	DESCRIPTION
---------	-------------	-------------

0	0x0	uImage header, header size: 64 bytes, Header CRC: 0x7DDB7CAD, created: 2022-07-23 16:09:43, image size: 136677 bytes, Data Address: 0xA0201000, Entry Point: 0xA0201000, Data CRC: 0x1B83F99B, OS: Linux, CPU: MIPS, image type: Standalone Program, compression type: none, image name: "Breed MT7621"
24213	0x5E95	JB00T STAG header, image id: 7, timestamp 0x5004418, image size: 1074814976 bytes, image JB00T checksum: 0x218, header JB00T checksum: 0x2100
27552	0x6BA0	Copyright string: "Copyright (C) 2022 HackPascal <hackpascal@gmail.com>"
28220	0x6E3C	LZMA compressed data, properties: 0x6D, dictionary size: 33554432 bytes, uncompressed size: 373234 bytes

- 14.2. Проверяем собранный файл Breed с помощью dumpimage -l.

Если вывод вот такой:

```
dumpimage -l breed-mt7621-xiaomi-r3g-En.bin
```

```
Image Name: Breed MT7621
```

```
Created: Sat Jul 23 12:09:43 2022
```

```
Image Type: MIPS Linux Standalone Program (uncompressed)
```

```
Data Size: 136677 Bytes = 133.47 KiB = 0.13 MiB
```

```
Load Address: a0201000
```

```
Entry Point: a0201000
```

Всё с вероятностью приближенной к 100% хорошо!

Если вывод вот такой:

```
dumpimage -l breed-mt7621-xiaomi-r3g-En.bin
```

```
Truncated file
```

```
dumpimage: cannot detect image type
```

То, наверное, в цепочке подсчёта размеров/CRC допущена оплошность/опечатка. Всё еще раз тщательно проверяем!

- 14.3. Для пущей важности еще раз производим отчасти раздел **10. Тестирование, проверка результата** и если ошибок/опечаток не найдено и Breed выводит сообщение при попытке прошить Bootloader без иероглифов красного цвета с табличкой, содержащей слева снизу латинские буквы MD5, то можно прошиваться!

Breed Web 恢复控制台

更新确认

文件已上传，请确认下方列出的信息

升级类型	常规固件
自动重启	是
类型	Bootloader
文件名	breed-mt7621-xiaomi-r3g-En.bin
大小	133.54KB
MD5 校验	324f0dec458ebd972d036bf11646cdce
闪存布局	小米 R3G 原厂

更新



Breed Web 恢复控制台

操作正在进行

您选择的操作正在进行
正在更新固件，请耐心等待至进度条完成

更新完成。设备正在重启。本页面不会刷新，请手动检查设备状态。

警告：在操作进行过程中请不要断开电源

15. Список книг на лето

- 15.1. **Success on hacking, extracting, modifying (Translating) and repacking BREED Bootloader!**
<https://forum.openwrt.org/t/success-on-hacking-extracting-modifying-translating-and-repacking-breed-bootloader/137710>
- 15.2. **U-Boot modification for MT7621 Devices**
<https://github.com/pinney/MT7621-u-boot-mod/tree/master>
- 15.3. **Breed bootloader breed-mt7621-xiaomi-r3g.bin reset button patcher**
<https://github.com/legale/breed-mt7621-xiaomi-r3g.bin-reset-button-changer>
- 15.4. **Binwalk – Полное руководство по инструменту для извлечения образа прошивки**
<https://forensicanvil.ru/forum/topic/tools/binwalk-instrumentu-izvlecheniya>
- 15.5. **Реверс-инжиниринг домашнего роутера с помощью binwalk**
<https://habr.com/ru/articles/487406/>
- 15.6. **История одного маленького реверс-инжиниринга или как мы BREED для Beeline Smartbox FLASH/GIGA расковыряли**
<https://habr.com/ru/articles/649039/>
- 15.7. **Реверс-инжиниринг домашнего роутера с помощью binwalk. Доверяете софту своего роутера?**
<https://habr.com/ru/articles/487406/>